Automotive Safety Gateway

QNX RTOS 8.0 on Raspberry Pi 4

USB SSD Bring-Up without UAS Support

Document ID: ASG-QNX-RPI4-001 | Revision: A

Pratik Suryawanshi
pratik.suryawanshi@uni-bayreuth.de

April 16, 2026

Abstract

This document records the initial bring-up of the Automotive Safety Gateway platform using QNX Real-Time Operating System (RTOS) 8.0.0 on a Raspberry Pi 4 Model B. The system employs a 64-bit AArch64 architecture with boot media on an external USB 3.0 SSD. The configuration addresses specific hardware challenges including UART clock stability via VPU core frequency locking, UAS protocol bypass for consumer-grade USB-SATA bridges, and deterministic CAM-based block device enumeration. Known deviations from automotive production requirements are documented for future hardware revisions.

1 System Architecture Overview

Table 1: Platform Component Summary

Component	Specification
Target Hardware	Raspberry Pi 4 Model B
Root Operating System	QNX RTOS 8.0.0 (AArch64)
Boot Media	External USB 3.0 SSD (Ankeje/JMicron bridge)
Debug Interface	FTDI USB-to-TTL Serial Bridge (3.3V Logic)
Edge Node (Pending)	STM32F411CEU6 (bare-metal/FreeRTOS via UART3)

The primary debugging interface uses the Pi's secondary UART (`miniUART`) mapped to GPIO pins 14 (TXD) and 15 (RXD). This choice avoids contention with the Bluetooth subsystem which normally occupies the primary PL011 UART.

2 Phase 0: Hardware Bootstrap & Payload Staging (via Raspberry Pi OS)

Before the QNX microkernel can be executed, the Raspberry Pi 4's onboard EEPROM and bootloader must be explicitly configured to boot from an external USB 3.0 SSD, bypassing the native JMicron UAS conflicts. The following procedure is performed from a temporary Raspberry Pi OS environment.

2.1 Base OS Initialization

1. Boot the Pi 4 using a standard microSD card flashed with Raspberry Pi OS.
2. Connect the target SSD (via Ankeje/JMicron adapter) to the Pi's USB 3.0 port.

2.2 Firmware UAS Quirk Application

To prevent the Pi's Linux kernel (and subsequent bootloader stages) from hanging on the JMicron bridge, append the hardware VID:PID quirk to the front of `/boot/firmware/cmdline.txt`:

```
usb-storage.quirks=152d:0576:u
```

Listing 1: USB Storage Quirk Entry in cmdline.txt

Explanation: The quirk string `152d:0576:u` identifies the JMicron bridge (Vendor ID `0x152d`, Product ID `0x0576`) and forces the `usb-storage` driver to fall back to the Bulk-Only Transport (BOT) protocol instead of UAS.

2.3 EEPROM Update (USB Boot Enablement)

Force the Pi 4 to update its onboard EEPROM to the latest stable release, ensuring USB mass storage boot is prioritized and stable:

```
sudo rpi-eeprom-update -d -a
sudo reboot
```

Listing 2: Raspberry Pi EEPROM Update Commands

The flags `-d -a` apply the default (latest stable) bootloader configuration automatically.

2.4 QNX Payload Staging

Transfer the compiled QNX Initial File System (IFS) binary from the host development machine to the Pi's FAT32 boot partition. Crucially, force a filesystem sync to ensure the SSD write buffers are flushed before powering down.

```
sudo cp ~/ifs-rpi4.bin /boot/firmware/ifs-rpi4.bin
sudo sync
```

Listing 3: Staging the QNX IFS Payload

2.5 Handoff Preparation

Shut down the Raspberry Pi OS cleanly. Remove the microSD card. The system is now hardware-staged to boot directly from the SSD using the custom `config.txt` parameters outlined in Section 4.

3 Hardware Debug Wiring (FTDI Console)

Table 2: FTDI to Raspberry Pi GPIO UART Connection

Raspberry Pi	Signal	FTDI USB-TTL
Pin 6 (GND)	GND	GND
Pin 8 (GPIO 14 / TXD)	TXD	RX
Pin 10 (GPIO 15 / RXD)	RXD	TX

- **Host PC Configuration:** `minicom` @ 115200 baud, 8N1, Hardware Flow Control **OFF**.

4 Raspberry Pi Bootloader Configuration

The Broadcom bootloader configuration file `/boot/firmware/config.txt` is adjusted to ensure stable UART clocking and proper hand-off to the QNX Initial File System (IFS). **Crucially**, the `dtoverlay=disable-bt` directive is *not* used to prevent pinmux conflicts.

```
# ==========================================
# QNX 8.0 AMP Platform Configuration
# ==========================================
# 1. Force AArch64 execution
arm_64bit=1

# 2. Enable miniUART and route it to GPIO 14/15
enable_uart=1

# 3. Lock the VPU core clock to 500MHz to stabilize miniUART baud rate
core_freq=500
core_freq_min=500

# 4. Show bootloader logs before kernel handoff (Debug only)
uart_2ndstage=1

# 5. Boot payload handoff
kernel=ifs-rpi4.bin
```

Listing 4: /boot/firmware/config.txt

- `core_freq=500` locks the VPU core clock to 500 MHz, eliminating baud rate drift on the mini-UART derived from this clock.
- `kernel=ifs-rpi4.bin` designates the QNX kernel image as the boot payload.

5 QNX System Builder Configuration (`rpi4.build`)

5.1 Kernel Startup & Console Routing

The QNX startup arguments explicitly bind the miniUART base address, reference clock, and baud rate to bypass Bluetooth/PL011 conflicts.

```
[virtual=aarch64le,raw -compress] boot = {
    # Explicitly map miniUART base address, clock (500MHz), and baud (115200)
    startup-bcm2711-rpi4 -v -D miniuart.0xfe215000.500000000.115200 -u reg -r 0,4096,1
    PATH=/proc/boot:/sbin:/bin:/usr/bin:/usr/sbin:/usr/libexec LD_LIBRARY_PATH=/proc/boot:/lib:/
        usr/lib:/lib/dll:/lib/dll/pci procnto-smp-instr -v
}
```

Listing 5: Kernel Startup Argument (excerpt)

5.2 Serial Driver Initialization

The miniUART device driver `devc-serminiuart` is started with parameters matching the kernel startup.

```
    ######################################################################
    ## Console SERIAL driver
    ######################################################################
    display_msg "Starting serial driver (/dev/ser1)"
    devc-serminiuart -b115200 -c500000000 -e -F -u1 0xfe215000,125
    waitfor /dev/ser1
    reopen /dev/ser1
```

Listing 6: Serial Driver Initialization (Buildfile Snippet)

5.3 USB Mass Storage & UAS Quirk Bypassing

Consumer-grade JMicron USB-to-SATA bridges (VID: `0x152d`, PID: `0x0576`) fail to enumerate under QNX's UAS stack. The `devb-umass` driver is instructed to bypass UAS and use the Common Access Method (CAM). Note that QNX exposes the USB block device as `/dev/hd0`, not `/dev/sd0`.

```
    # Start the Mass Storage driver and bypass UAS for JMicron
    echo "Starting USB Storage (UAS Bypass)..."
    devb-umass cam pnp noslow umass prot=umass wait=30

    # Wait for the CAM subsystem to expose the physical hard disk block
    waitfor /dev/hd0 30
```

Listing 7: USB Mass Storage Startup Script (`usb_start.sh`)

6 Known Architectural Deviations from Production Intent

1. **Non-Automotive Boot Timing:** The 30-second `wait` state for USB SuperSpeed enumeration and block device ready is incompatible with typical automotive ISO 26262 requirements (< 2 seconds). For production (Rev B), the boot media will be migrated to industrial eMMC or SPI NOR flash to achieve millisecond-level enumeration.

2. **Volatile `/var` Mounting:** Until a QNX6 Power-Safe file system partition is formatted on the target drive, the `.storage-server.sh` script creates a RAM disk (`devf-ram`) to host `/var`. This allows networking and SSH daemons to initialize but results in non-persistent logs and state across reboots.

7 Conclusion

The current bring-up successfully demonstrates a stable QNX 8.0 serial console and bootable root filesystem on Raspberry Pi 4 hardware using a USB SSD. The documented workarounds for UART clock drift and UAS enumeration provide a functional development baseline. Future hardware revisions will address the identified deviations to align with automotive production standards.